

UNIVERSIDADE FEDERAL DE PELOTAS
Centro de Desenvolvimento Tecnológico
Programa de Pós-Graduação em Computação



Exame de Qualificação

Um Blabla Blablabla com Aplicações em Blablabla

Marilton Sanchotene de Aguiar

Pelotas, 2025

Marilton Sanchotene de Aguiar

Um Blabla Blablabla com Aplicações em Blablabla

Exame de Qualificação apresentado ao Programa de Pós-Graduação em Computação do Centro de Desenvolvimento Tecnológico da Universidade Federal de Pelotas, como requisito parcial à obtenção do título de Doutor em Ciência da Computação.

Orientador: Prof. Dr. Marilton Sanchotene de Aguiar
Coorientador: Prof. Dr. Marilton Sanchotene de Aguiar
Colaborador: Prof. Dr. Marilton Sanchotene de Aguiar

Pelotas, 2025

Dedico...

AGRADECIMENTOS

Agradeço...

Só sei que nada sei.

— SÓCRATES

RESUMO

AGUIAR, Marilton Sanchotene de. **Um Blabla Blablabla com Aplicações em Blablabla**. Orientador: Marilton Sanchotene de Aguiar. 2025. 24 f. Exame de Qualificação (Doutorado em Ciência da Computação) – Centro de Desenvolvimento Tecnológico, Universidade Federal de Pelotas, Pelotas, 2025.

Bla blabla blablalba bla. Bla blabla blablalba bla. Bla blabla blablalba bla. Bla
blabla blablalba bla. Bla blabla blablalba bla. Bla blabla blablalba bla. Bla blabla
blablalba bla. Bla blabla blablalba bla. Bla blabla blablalba bla. Bla blabla blablalba bla.
Bla blabla blablalba bla. Bla blabla blablalba bla. Bla blabla blablalba bla. Bla blabla
blablalba bla. Bla blabla blablalba bla. Bla blabla blablalba bla. Bla blabla blablalba bla.
Bla blabla blablalba bla. Bla blabla blablalba bla. Bla blabla blablalba bla. Bla blabla
blablalba bla.

Palavras-chave: palavrachave-um; palavrachave-dois; palavrachave-tres; palavrachave-quatro.

ABSTRACT

AGUIAR, Marilton Sanchotene de. **Titulo do Trabalho em Ingles**. Advisor: Marilton Sanchotene de Aguiar. 2025. 24 f. Qualification Exam (Doctorate in Computer Science) – Technology Development Center, Federal University of Pelotas, Pelotas, 2025.

Bla blabla blablabla bla. Bla blabla blablabla bla. Bla blabla blablabla bla. Bla
blabla blablabla bla. Bla blabla blablabla bla. Bla blabla blablabla bla. Bla blabla
blablabla bla. Bla blabla blablabla bla. Bla blabla blablabla bla. Bla blabla blablabla bla.
Bla blabla blablabla bla. Bla blabla blablabla bla. Bla blabla blablabla bla. Bla blabla
blablabla bla. Bla blabla blablabla bla. Bla blabla blablabla bla. Bla blabla blablabla bla.
Bla blabla blablabla bla. Bla blabla blablabla bla. Bla blabla blablabla bla. Bla blabla
blablabla bla.

Keywords: keyword-one; keyword-two; keyword-three; keyword-four.

LISTA DE FIGURAS

Figura 1	Original map.	17
Figura 2	Original map sampling.	18
Figura 3	Trajectories generated by the A* algorithm.	19
Figura 4	Screenshot showing an obstacle being encountered	19

LISTA DE TABELAS

Tabela 1	Performance comparison of the A* algorithm	18
----------	--	----

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
NUMA	Non-Uniform Memory Access
SIMD	Single Instruction Multiple Data
SMP	Symmetric Multi-Processor
SPMD	Single Program Multiple Data

SUMÁRIO

1	INTRODUÇÃO	12
2	RELATED WORK	14
3	PROPOSTA DE PESQUISA	16
3.1	CONCLUSIONS AND FUTURE WORK	20
4	CRONOGRAMA	21
	REFERÊNCIAS	22
	APÊNDICE A UM APÊNDICE	24

1 INTRODUÇÃO

As robots become increasingly present in human environments, the importance of multi-robot communication will continue to grow(?). For this communication to be effectively utilized in practical applications, it must meet certain essential requirements. One of the most critical requirements is energy efficiency, given that robots are autonomous entities with their own independent power sources. However, these power sources have a limited duration and therefore require frequent recharging(?).

Existem duas principais maneiras de economizar energia em robôs: com melhor hardware e melhor software(?). Enquanto a primeira está fora do escopo deste artigo, buscamos investigar propostas que possam contribuir com o segundo aspecto. Uma das maiores fontes de consumo de energia de robôs é na sua localização e cálculo de trajetória(?). Além disso, embora possa não ser tão significativo, a necessidade frequente de comunicação também envolve o consumo de energia de maneira que poderia ser desejável minimizar a comunicação em ambientes com múltiplos robôs. Além disso, uma comunicação frequente exige condições de transmissão que nem sempre podem estar disponíveis, de maneira que os algoritmos de um robô não podem ser totalmente dependentes desta(?)(?).

In this context, we propose a communication method inspired by board games, which has the potential to aid in both of these aspects.

There are countless board games, but a significant portion of them are characterized by an environment where pieces move within a discrete space(?). This discretization of space inspires intriguing algorithmic approaches that can be both easily understood by humans and quickly processed by machines. The use of simple algorithms can not only reduces energy consumption but also takes advantage of discretized environments as a simplification of reality, which can lead to a reduced need for data transfer.

Among board games, one of the oldest and most well-known is Chess(?). As a game typically played by two people, it was natural that soon after its invention, the desire to play the game with two players who are physically distant from each other would arise. Given that both players are familiar with the initial state and the rules of

the game, it is possible to determine the future state of the board based on the history of moves. A standard was thus established that allows for the encoding of only the piece being moved and its destination(?).

De maneira análoga, o presente artigo apresenta um mecanismo semelhante com o intuito de compartilhar informações sobre a ocupação do ambiente minimizando a quantidade de dados trocados entre os robôs com uma consequente diminuição de energia.

2 RELATED WORK

Várias abordagens tem sido utilizadas para obter um melhor uso de recursos energéticos. (?) trata o conjunto de robôs como uma espécie de esquadra que podem compartilhar informações sobre trajetórias. Outra proposta diferente porém com o mesmo objetivo é a dos "robôs caroneiros" (?). Aqui um robô avisa através de um meio compartilhado qual o seu destino e outros robôs podem simplesmente seguir o primeiro. Entretanto para maior aproveitamento, essas abordagens requerem que os robôs estejam se deslocando para um mesmo local no mapa.

O artigo de (?) faz algumas modificações em um algoritmo do tipo "colônia de formigas" para transformar este em multi-objetivo, sendo a minimização do consumo de energia um destes.

(?) enfatiza o treinamento distribuído de uma rede neural com o objetivo de gerar um mapa de uma localização desconhecida mas nele não é dada muita ênfase na comunicação entre os agentes. (?) também tem uma abordagem baseada em vários agentes distribuídos que compartilham o objetivo de encontrar focos de incêndio. Aqui os autores optaram por utilizar um mapa topológico compartilhado.

Diversos projetos como (?) provem conjuntos de dados que podem ser utilizados para treinamentos e validação de soluções envolvendo multi-robôs. (?) propõe não só um conjunto de dados para validação como uma plataforma para o desenvolvimento de aplicações envolvendo robôs e outros agentes móveis, incluindo pessoas.

(?) faz uma interessante mescla entre grades tridimensionais armazenadas individualmente por cada robô e um mapa topológico compartilhado. Conforme o mesmo a abordagem parece bastante interessante em ambientes desconhecidos de larga escala. A proposta faz uso de pequenos pacotes de dados para compartilhar informações mesmo com baixa largura de banda. Outro sistema desenvolvido com base em exploração de sistemas desconhecidos é (?). O sistema é muito promissor embora os próprios autores citem que a arquitetura possui uma forte característica centralizadora que pode ser retrabalhada posteriormente. Outro projeto que mostra preocupação com baixa comunicação entre os agentes é (?), nele os

Ainda sobre a representação de mapas, alguns artigos focam em como múltiplos

mapas podem ser fundidos em um único(?). Diferente da proposta do nosso artigo, aqui os mapas são criados em separado e depois mesclados. Novamente a abordagem baseada em jogos de tabuleiro mostra-se interessante pois se os mapas são criados a partir de uma mesma base e mantidos sincronizados a mesclagem entre esses mapas é mais simples. Claro que essa abordagem só é possível com ambientes previamente mapeados.

Uma das primeiras decisões a serem tomadas no projeto de robôs móveis é a escolha de como o mapa do mundo será representado. As duas principais maneiras de fazer isso são utilizar um mapa baseado em grade ou um mapa topológico(?). No mapa em forma de grid o espaço físico, seja bi ou tridimensional é dividido em células de tamanho relativamente uniformes. No mapa topológico o ordenamento é dado pelas características de um determinado ambiente. Pode-se por exemplo representar espaços de acordo com a interconexão entre eles, sem levar em conta a área dos mesmos. Cada abordagem tem suas vantagens e desvantagens e inclusive é possível, na prática, usar uma mescla de ambas(?).

No artigo (?) é apresentado o algoritmo RRTX, que aparenta ser bastante adequado para atualização de trajetórias em ambientes sujeitos à mudanças frequentes. Porém não é mencionado no mesmo preocupação explícita com o consumo de energia.

3 PROPOSTA DE PESQUISA

Para testar a proposta foi utilizado como middleware o software ROS (Robot Operating System). ROS foi escolhido pois é um middleware bastante utilizado na área de robótica (??) e que possui um ecossistema bastante atuante que provê exemplos e bibliotecas de códigos diversos. O ROS suporta várias linguagens sendo incentivadas no site oficial da plataforma as linguagens C++ e Python. Para agilizar a prototipagem foi adotada a linguagem Python para o projeto.

Além disso o ROS possui dois principais mecanismos de comunicação: o modelo cliente/servidor e o modelo publisher/subscriber e ambos foram utilizados na proposta. A imagem ?? mostra a estrutura básica de comunicação do sistema. As elipses representam nós na arquitetura, quadrados representam tópicos e losângos representam serviços. As setas indicam o fluxo de informação.

O nó "Map Share Server" é responsável por iniciar antes dos demais e fazer o carregamento do mapa. Esse nó provê os serviços GetMapData e SendMsgServer, sendo que o primeiro serviço é responsável por fornecer os dados do mapa para os clientes e o outro é responsável por receber mensagens vindas dos clientes. Além disso o nó também escreve no tópico GetMapInfo a cada vez que houver alguma mudança no mapa.

O nó "Map Client" representa os robôs movimentam-se no cenário e compartilhando informações. Ao iniciar os nós desse tipo chamam o serviço GetMapData para obter informações sobre o mapa. Quando os clientes encontram obstáculos no mapa eles fazem uma chamada ao serviço SendMsgServer passando a localização do obstáculo. O serviço recebe o aviso e escreve no tópico "GetMapInfo" informando todos os nós do obstáculo encontrando. Os nós inscrevem-se no tópico "GetMapInfo" logo após terem iniciado e portanto recebem mensagens sempre que alguém, no caso o servidor, publicar nesse nó. Ao receber atualizações os nós também atualizam internamente seus mapas para ficarem cientes dos obstáculos e levarem eles em consideração ao calcularem suas trajetórias.

O nó cliente também é capaz de ler o teclado e portanto aceitar comandos, sendo os principais 'goto', 'gotominimap', 'put', 'exit'. O comando 'goto' e 'gotominimap' são

responsáveis por iniciar uma movimentação automática até algum ponto do mapa e serão detalhados a seguir. 'put' aceita coordenadas e simula a colocação de obstáculos ainda não detectados. O comando 'exit' encerra o cliente. Além disso comandos de movimentação através das teclas 'w', 's', 'a' e 'd' também são aceitos.

Tanto os comandos 'goto' quanto 'gotominimap' aceitam dois parâmetros x e y que representam o destino para onde o agente deve ser movido. Ambos os comandos utilizam o algoritmo A* para o cálculo de trajetórias. A diferença entre ambos é que enquanto o comando 'goto' utiliza como entrada do algoritmo o mapa completo, o 'gotominimap' utiliza como entrada uma amostragem do mapa. Essa abordagem é utilizada também com o intuito de consumir menos energia no cálculo da trajetória.

A imagem 1 mostra um mapa criado para testar o conceito. As cores, no momento, não tem significado especial, mas foram adotadas pois futuramente pretende-se adicionar um significado semântico no mapa, onde cores diferentes representariam salas com propósitos diferentes.

A figura 2 representa uma versão amostrada do mesmo mapa. A versão original do mapa possui um tamanho quadrado de altura de 720 pixels. Para a amostragem foi escolhido um fator de 1:20. Para melhor entendimento da proposta, no artigo ela foi redimensionada por um software externo para obter um tamanho semelhante ao mapa original.

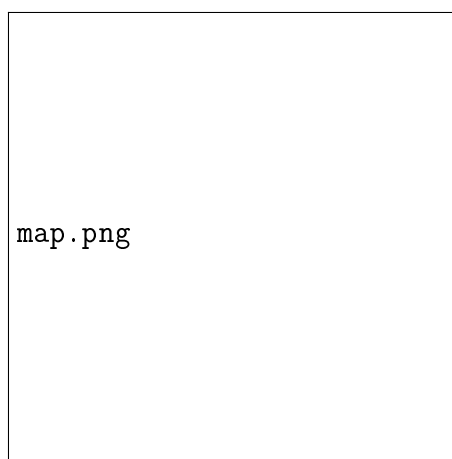


Figura 1 – Original map.

A imagem amostrada possui na realidade um tamanho quadrado com 72 pixels de altura. Com o fator escolhido seria esperado que o quadrado possuísse dimensões 36 x 36 pixels. Porém

Conforme dito anteriormente os comandos 'goto' e 'gotominimap' levam a uma simulação de movimento do robô. Em versões preliminares o mapa, trajetória e demais elementos foram representados em modo texto. Para facilitar a visualização foi adotada uma visualização gráfica do mapa, trajetória e posição do agente. Foi utilizada a biblioteca Pygame que permite gerenciar de forma mais fácil o desenho de formas

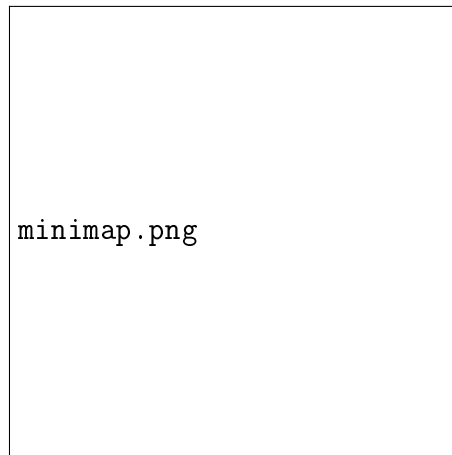


Figura 2 – Original map sampling.

geométricas, bem como utilizar as vantagens da biblioteca na criação de janelas, atualização da tela e captura de eventos do teclado e mouse.

A posição original do agente é caracterizada por um quadrado vermelho e o destino do mesmo é marcado por um quadrado de cor ciano. Antes de começar a se movimentar o agente utiliza o algoritmo A* para calcular qual a melhor trajetória possível. O A* é o algoritmo mais famoso de planejamento de trajetórias e é computacionalmente simples e portanto muito adequado para situações onde deseja-se economizar energia (?). Segundo (?) "A* is a strong-performing option for static environments because of the low memory usage, computational speed, less implementation complexity, and efficiency making it suitable also for use in embedded system deployment". A heurística utilizada na implementação foi a distância Manhattan.

A imagem 3 mostra o caminho calculado utilizando o mapa original na cor verde e o caminho calculado para o mesmo destino utilizando a amostragem do mapa na cor vermelha. A imagem foi ampliada para facilitar a comparação das trajetórias. É possível observar que a trajetória que leva em consideração o mapa completo é bem menos retilínea, fato que também levaria a um maior consumo de energia em robôs reais devido à necessidade de constantes ajustes na trajetória (?).

A tabela 1 mostra os valores médios de quantidade nós, e tempo (em milisegundos) obtidos após executar o algoritmo A* com o mapa completo e com o mapa amostrado com 10 destinos aleatórios diferentes.

Tabela 1 – Performance comparison of the A* algorithm

	Number of positions	Time (ms)
Complete map	434.8	0.044
Sampled map	22.8	0.026

Quando o comando 'gotomini' é digitado uma thread é iniciada e ela ficará en-

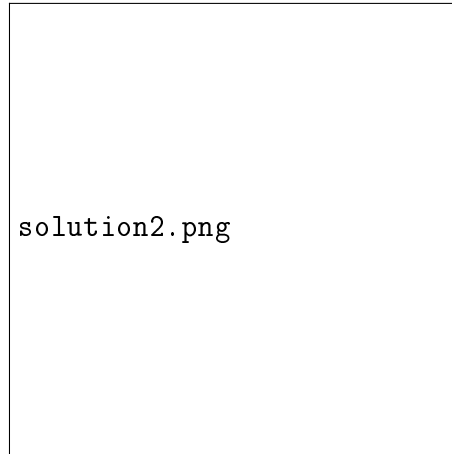


Figura 3 – Trajectories generated by the A* algorithm.

carregada de simular a movimentação do robô. Paralelamente a leitura do teclado é retomada. Assim é possível utilizar o comando 'put', que também aceita dois parâmetros x e y e utiliza-os para permitir a inclusão no simulador de um obstáculo que não estava anteriormente no mapa e portanto não foi levado em consideração no cálculo da trajetória.

A imagem 4 mostra uma captura de tela em que o agente (quadrado vermelho) está seguindo sua trajetória (linha vermelha), até que encontra um obstáculo adicionado manualmente (quadrado azul). No momento em que o mesmo é detectado a movimentação é interrompida e o servidor é avisado desta diferença entre o mapa conhecido anteriormente e o mapa real. A imagem também mostra parte do terminal no qual o log de execução é mostrado. Posteriormente o agente pode recalculá-la sua rota para atingir seu objetivo original com o mapa devidamente atualizado.

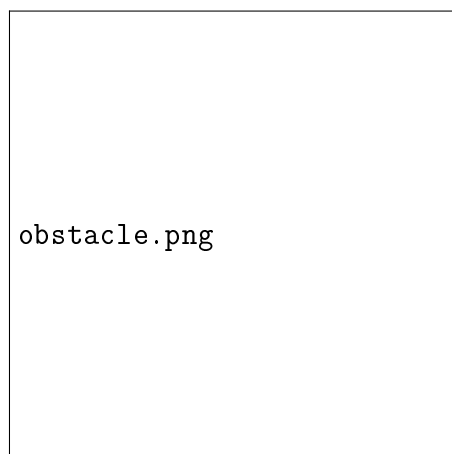


Figura 4 – Screenshot showing an obstacle being encountered

Na atuação implementação a mensagem é enviada ao servidor e então somente este escreve no tópico 'GetMapInfo' avisando todos os clientes da modificação. Assim cada cliente fica ciente do novo obstáculo e o leva em consideração ao calcular suas trajetórias. O ROS permite que o cliente escreva em um tópico mas isso não foi

feito pois versões futuras desse protótipo irão poderão levar em consideração algum algoritmo de verificação se o obstáculo foi reportado por uma quantidade mínima de agentes. Isso é interessante pois a detecção de obstáculo por um único agente pode tanto ser em decorrência de algum erro no sensoramento do mesmo quanto pode ter ocorrido em função de algum obstáculo móvel, que no instante seguinte pode ter mudado de posição no mapa.

Essa abordagem foi imaginada para ambientes previamente mapeados, tal qual um tabuleiro de xadrez, no qual apenas alguns objetos dentro do ambiente são capazes de se movimentar e portanto é possível compartilhar o mapa global do ambiente previamente. Entendemos que ambientes desconhecidos como por exemplo um ambiente de resgate, possuem requisitos maiores de transferência de dados pois é necessário compartilhar todo o mapa, que inicialmente é desconhecido. Embora isso limite a aplicação da presente proposta acreditamos que ela ainda é válida por possuir inúmeras aplicações, tal qual uma fábrica, campus universitário ou um shopping, no qual a planta do local é conhecida previamente e mudanças estruturais são mais raras sendo necessário somente mapear objetos menores e pessoas dentro dos mesmos e não todo o ambiente em si.

4 CRONOGRAMA

O cronograma vai aqui

5 CONCLUSIONS AND FUTURE WORK

Outras ações serão adicionadas na arquitetura como o compartilhamento da própria posição dos agentes que atualmente não é compartilhada.

Também serão adotadas outras estratégias para o consumo de energia como por exemplo realizar, quando houver conectividade a execução do A* no servidor, economizando energia no robô real.

Futuramente pretende-se integrar o software desenvolvido com simuladores existentes e que possuam mecanismos de detecção de colisão baseados na física e na simulação de sensores diversos tais como Lidars.

Por fim pretende-se implementar a solução em robôs reais e permitir que os mesmos possam desempenhar tarefas reais mesmo que inicialmente de pequena monta tais como carregar pequenos objetos ou servir de guias em locais públicos.

Acreditamos que mesmo com as limitações apontadas nas seções anteriores a ideia é promissora e foi mostrada viável no software que foi descrito neste artigo. Esperamos que essa ideia possa influenciar outros trabalhos a desenvolver soluções elegantes, simples de entender para seres humanos e com menor consumo de energia para agentes robóticos.

REFERÊNCIAS

AGUIAR, M.; MARILTON, A. **Título da Monografia**. 2005. 85p. Trabalho de Conclusão (Curso de Ciência da Computação) — Centro de Desenvolvimento Tecnológico, Universidade Federal de Pelotas, Pelotas.

MOORE, R. E. **Methods and Applications of Interval Analysis**. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 1979. xi + 190p.

NEUMANN, J. von; AGUIAR, M. **Theory of Self-Reproducing Automata**. [S.l.: s.n.], 1966. xix + 388p.

Apêndices

APÊNDICE A – Um Apêndice

Bla blabla blablabla bla. Bla blabla blablabla bla. Bla blabla